

Visualización de datos en *leaflet*

Jorge Juvenal Campos Ferreira



Programa interdisciplinario
de Ciencia de Datos

Junio 2024



Objetivos de la sesión

1. Que los estudiantes aprendan que es una **función**
2. Que los estudiantes aprendan qué es un **bucle**
3. Que los estudiantes aprendan a elaborar mapas en *leaflet*





Funciones en R



Para entender los procesos en **R**, son de utilidad dos eslógans:

Todo lo que existe es un objeto.

Todo lo que ocurre pasa por llamar una función.

--- John Chambers

Una función es **un objeto de R** que, al ser activado, lleva a cabo ciertos cálculos (computations) de manera estructurada con un fin o propósito.

Generalmente, un usuario de R principiante va a hacer uso de las funciones previamente programadas, tanto de la biblioteca base como de las distintas bibliotecas que puede instalar desde el CRAN.



Eventualmente, un usuario más experimentado se dará cuenta de que **muchas de las labores que lleva a cabo a través del código son muy repetitivas, o monótonas, o muy específicas, o bien, acciones que ya ha realizado antes**. Y entonces llegará el momento de programar sus propias funciones.

Una función nos permite romper el código de manera lógica, en procesos simples y estructurados que se vuelven sencillos de manejar y entender.

Crear funciones nos permite evitar duplicidades el en código y facilitar su lectura.

Funciones en R



La sintaxis para escribir funciones en R es la siguiente:

```
func_name <- function (argument) {  
  # statement  
  ...orden_1...  
  ...orden_2...  
  ...orden_3...  
}
```



Las funciones estan compuestas de cuatro cosas:

- 1) Los *argumentos* que controlan la manera en que llamamos a la función.
- 2) El *cuerpo* de la función, esto es, el código dentro de la función.
- 3) El *nombre* de la función (esto es opcional).
- 4) El ambiente (*environment*) o la estructura que determina la manera en que la función encuentra los elementos necesarios para funcionar. (En qué parte se alojan los objetos que va a utilizar).



Tips para realizar funciones:

- 1) Elabora primero el caso $n = 1$ (esto es, primero lleva a cabo el procedimiento para uno de los casos que vas a evaluar).
- 2) Una vez que funciona para $n = 1$, evalúa para $n = N$. Si brota un error, averigua qué excepción lo causó, y corrige la función.
- 3) Una vez que las excepciones están resueltas, pruebala ya dándole un uso real.
- 4) Implemente (si lo crees necesario) un administrador de errores (Error Handlers) en caso de que consideres de que otros usuarios puedan llegar a cometer errores que afecten el buen funcionamiento de tu función.

Ejemplo de administración de errores

```
a <- 1
b <- 3

suma <- function(a,b){

  tryCatch(r <- a + b,
           error = function(e){
             warning("Un argumento no es numerico!\n")
           })
  return(r)
}

# Buen funcionamiento
suma(a = 1, b = 1)
```

```
## [1] 2
```



Ejemplo de administración de errores

```
# Mal funcionamiento  
suma(a = 1, b = "poyo")
```

Error in suma(a = 1, b = "poyo") : object 'r' not found In addition: Warning message: In value[[3L]] (cond) : Un argumento no es numerico!



¿Qué es un bucle *for*?

- Un bucle *for* permite ejecutar un bloque de código repetidamente.
- Se utiliza para iterar sobre una secuencia de elementos como vectores, listas o cualquier objeto iterable.

Sintaxis del bucle *for*

```
for (variable in secuencia) {  
    # Código a ejecutar en cada iteración  
}
```

Ejemplo

```
# Imprimir números del 1 al 5  
for (i in 1:5) {  
    print(i)  
}
```



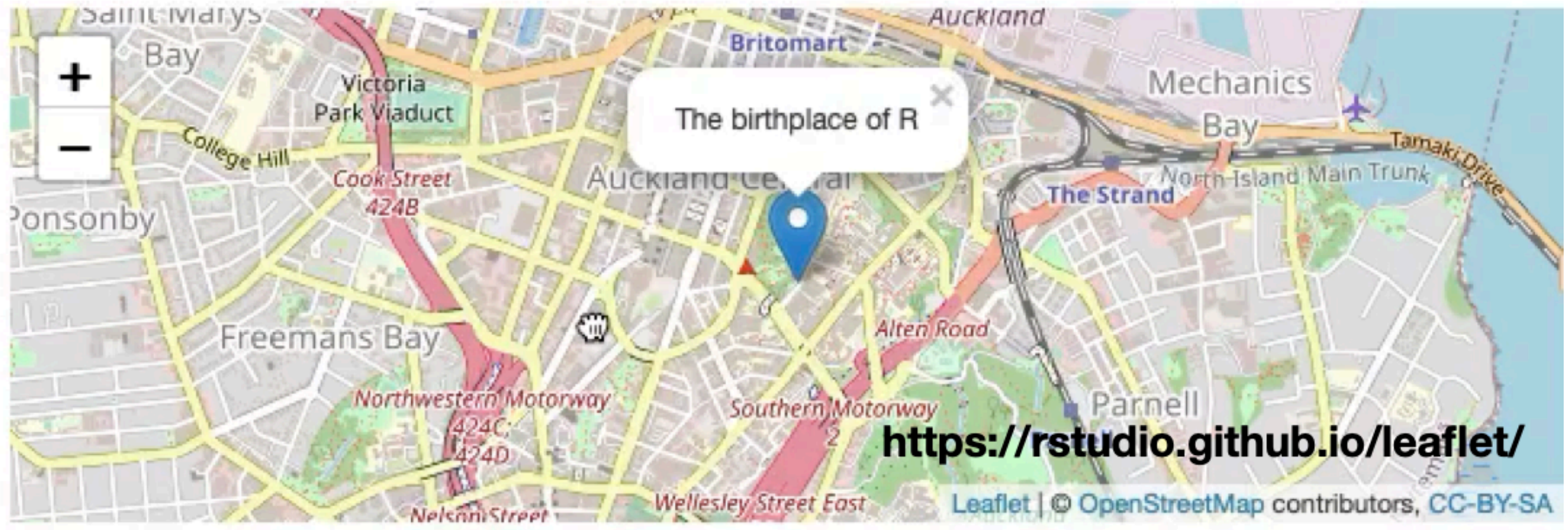
**Hacer ejercicios 1:3 de la sección
práctica**



Mapa interactivo



Un mapa que añade las funciones de interactividad

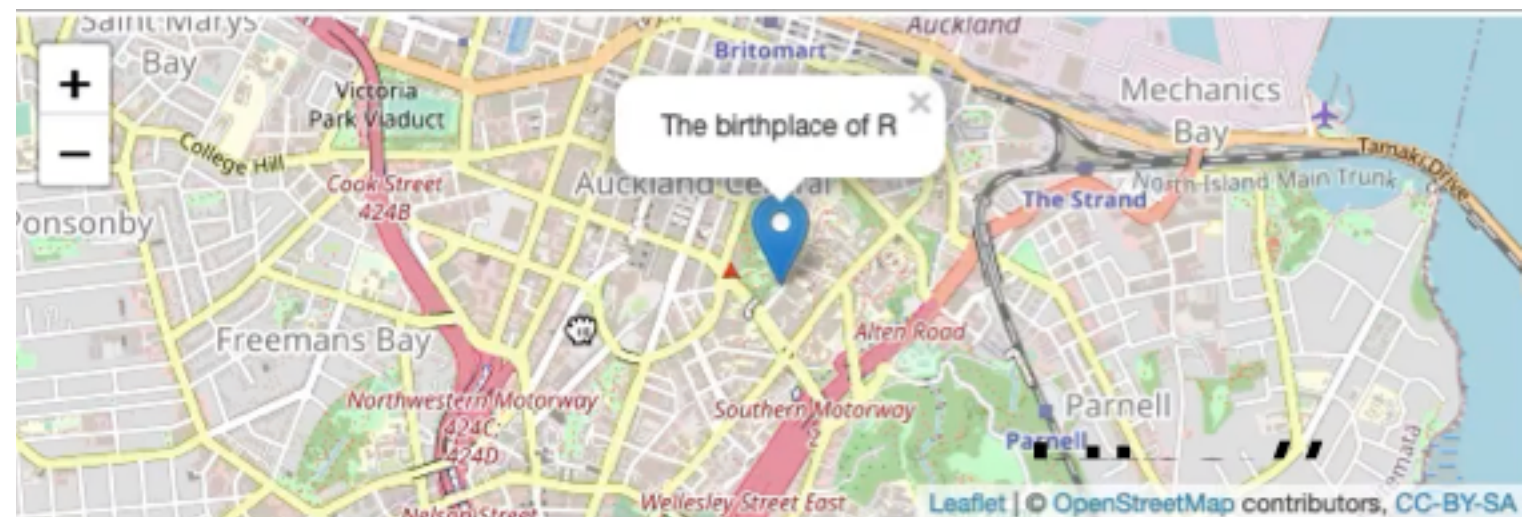


Casos de uso



Si voy a publicar el mapa en internet y:

- Quiero que el usuario pueda explorar los datos.
- Quiero plasmar muchas variables en un mapa.
- La información es muy densa y no se ve bien en presentación estática o en papel
- Voy a hacer una aplicación web



Librerías



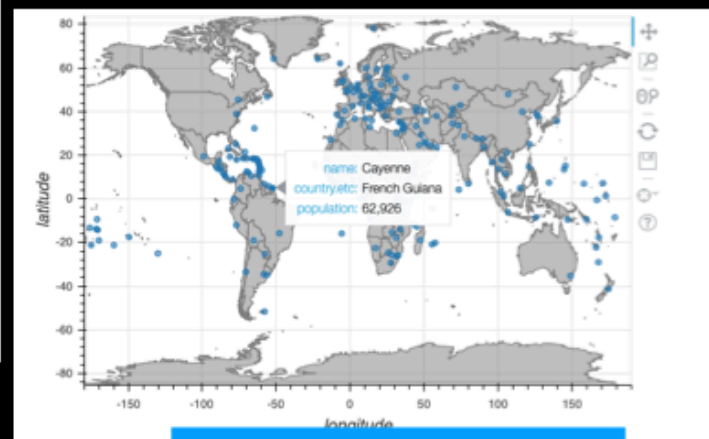
Leaflet, plotly, tmap, mapbox, highcharter, rbokeh, etc.

En esta sesión utilizaremos *leaflet*, por ser una de las más comunes para elaborar mapas interactivos.

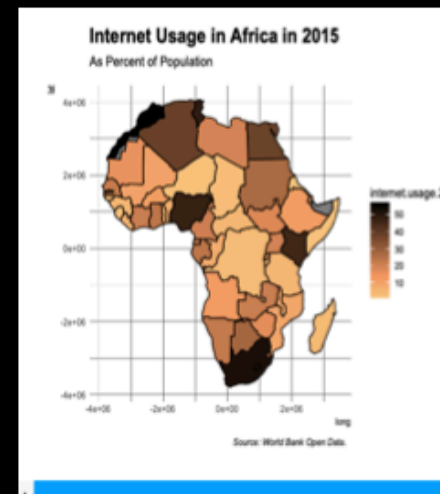
Más referencia en <https://bhaskarvk.github.io/user2017.geodativiz/notebooks/03-Interactive-Maps.nb.html>



plotly



rbokeh



ggraph



Leaflet

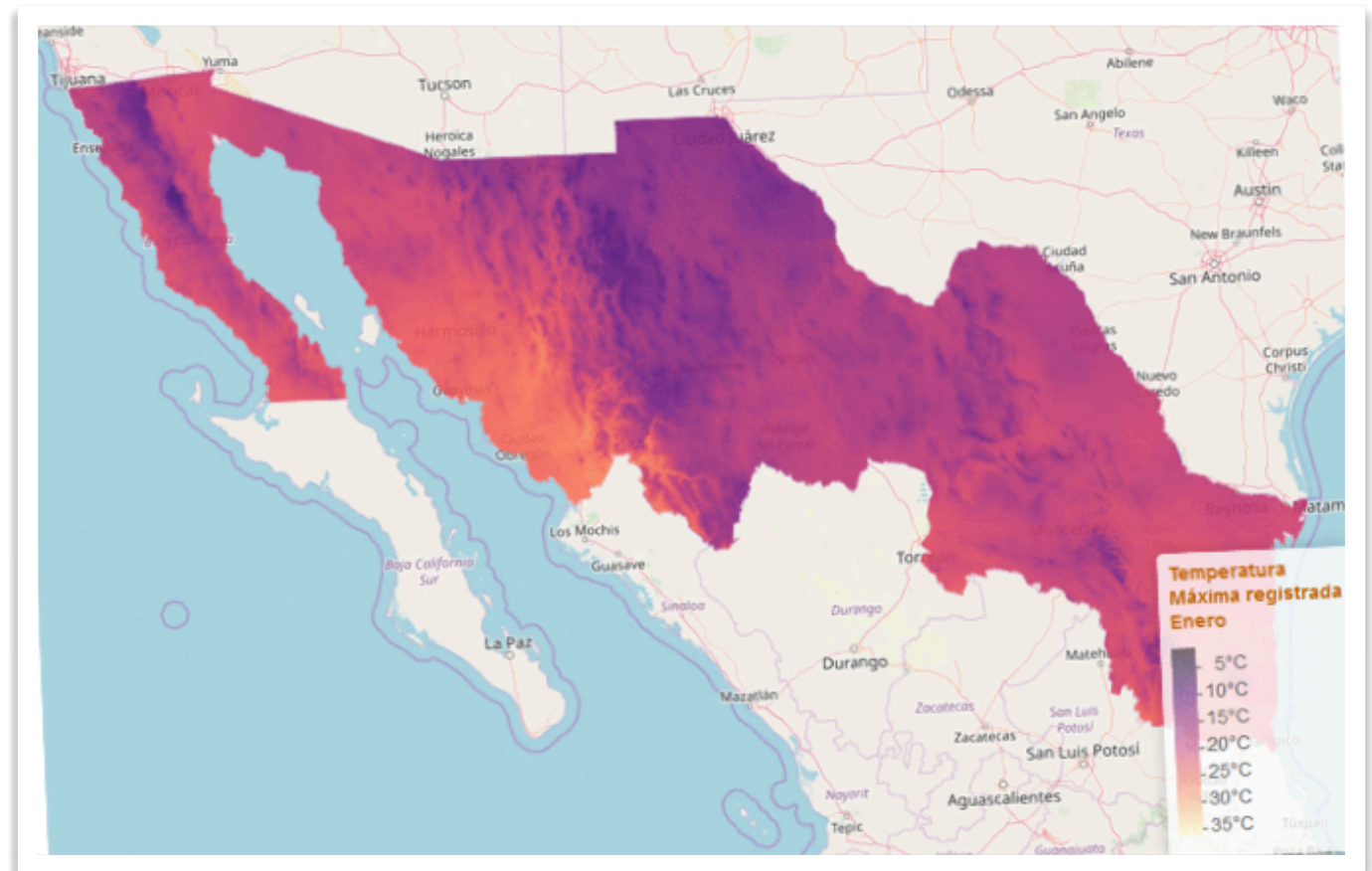
Librería `{leaflet}`



La librería *Leaflet.js* es una librería *open-source* de Javascript que nos permite crear mapas interactivos. La librería de R, `{leaflet}`, nos permite crear fácilmente mapas con esta librería utilizando la sintaxis de R, **sin tener que aprender javascript.**

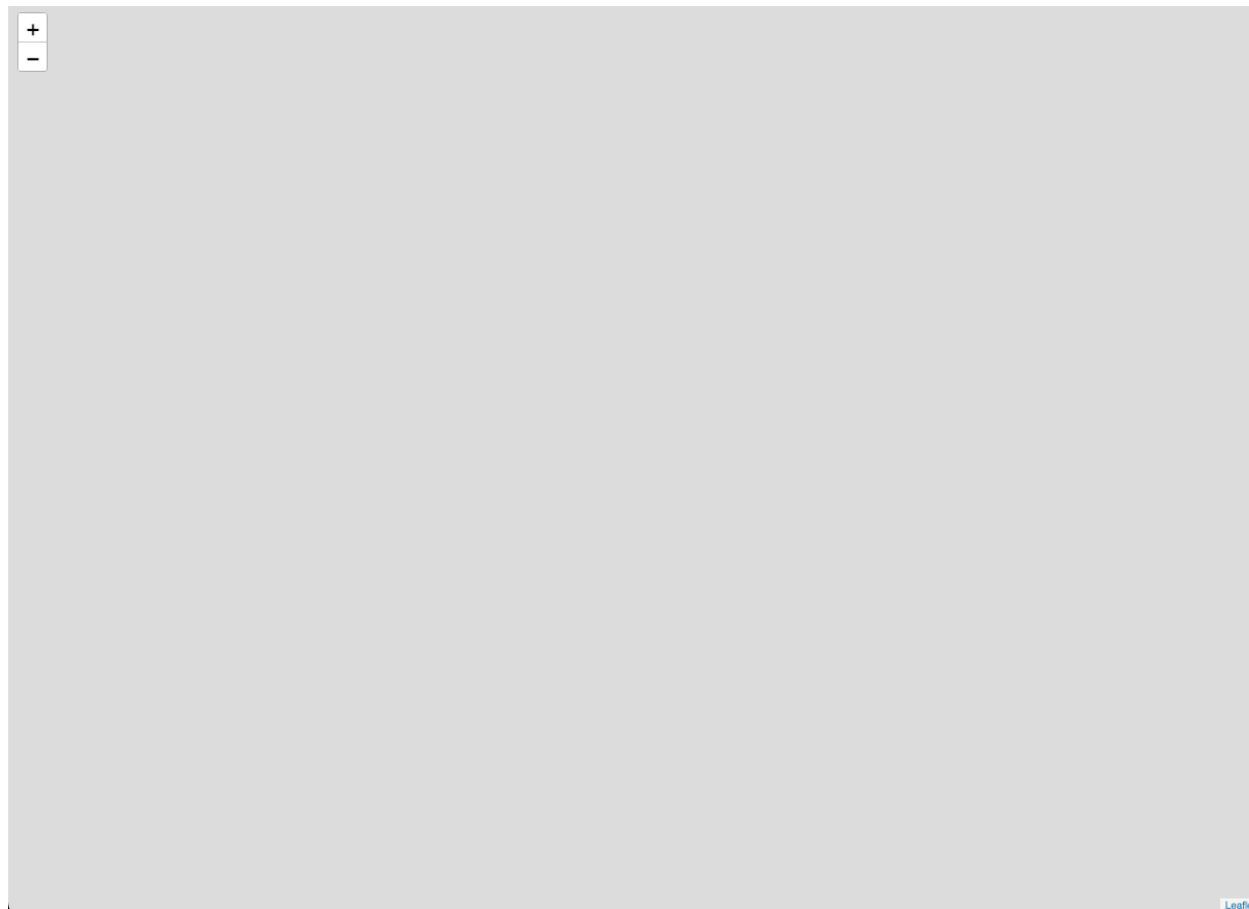
Referencia principal:

<https://rstudio.github.io/leaflet/>



**Hacer ejercicios de la sección
práctica del 4 en adelante.**

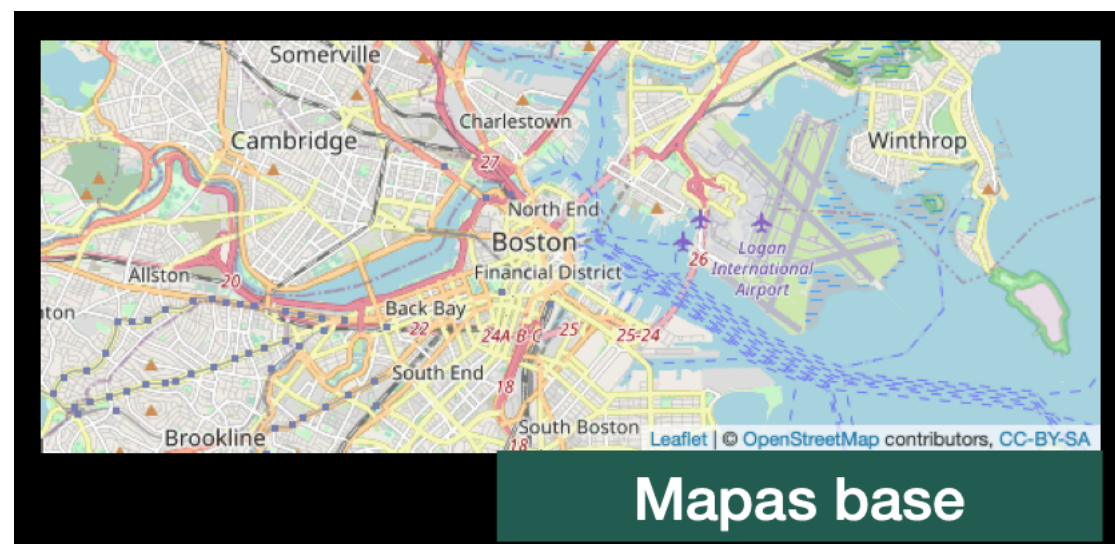
La función `leaflet::leaflet()` activa el lienzo para poder empezar a trabajar en leaflet. Igualmente, **es la función que recibe el objeto `sf` para hacer el mapa** y es la función en la cual se le van a ir poniendo las distintas opciones avanzadas (si se requieren), como quitar el Control de Zoom, definir el zoom mínimo y máximo o permitir el arrastre en el mapa.



Para añadir mapas base:

-addTiles(), agrega un mapa base default al mapa que estamos elaborando. (Nota: Recordar que leaflet trabaja con el crs = 4326, por lo que si no estamos trabajando con este crs van a salir cosas muy extrañas).

-addProviderTiles("nombre_del_mapa-base"), agrega un mapa base distinto al default. Para ver que mapas base hay disponibles hay que acceder al catálogo a través del objeto `leaflet::providernames` o visitar <https://leaflet-extras.github.io/leaflet-providers/preview/>



Para añadir Capas de geometrías:

-**addPolygon()**, para añadir polígonos.

-**addPolylines()**, para añadir líneas (polilíneas).

-**addCircleMarker()**, para añadir círculos en los puntos.

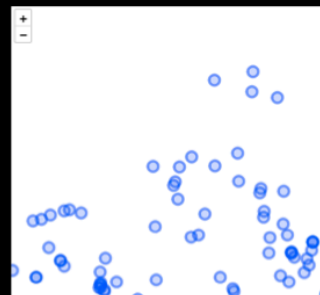
-**addMarker()**, para añadir marcadores sobre los puntos.

-**addRasterImage()**, para añadir imágenes raster.

Geometrías



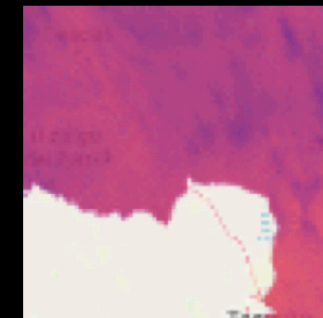
-Líneas



-Puntos



-Polígonos



-Rasters

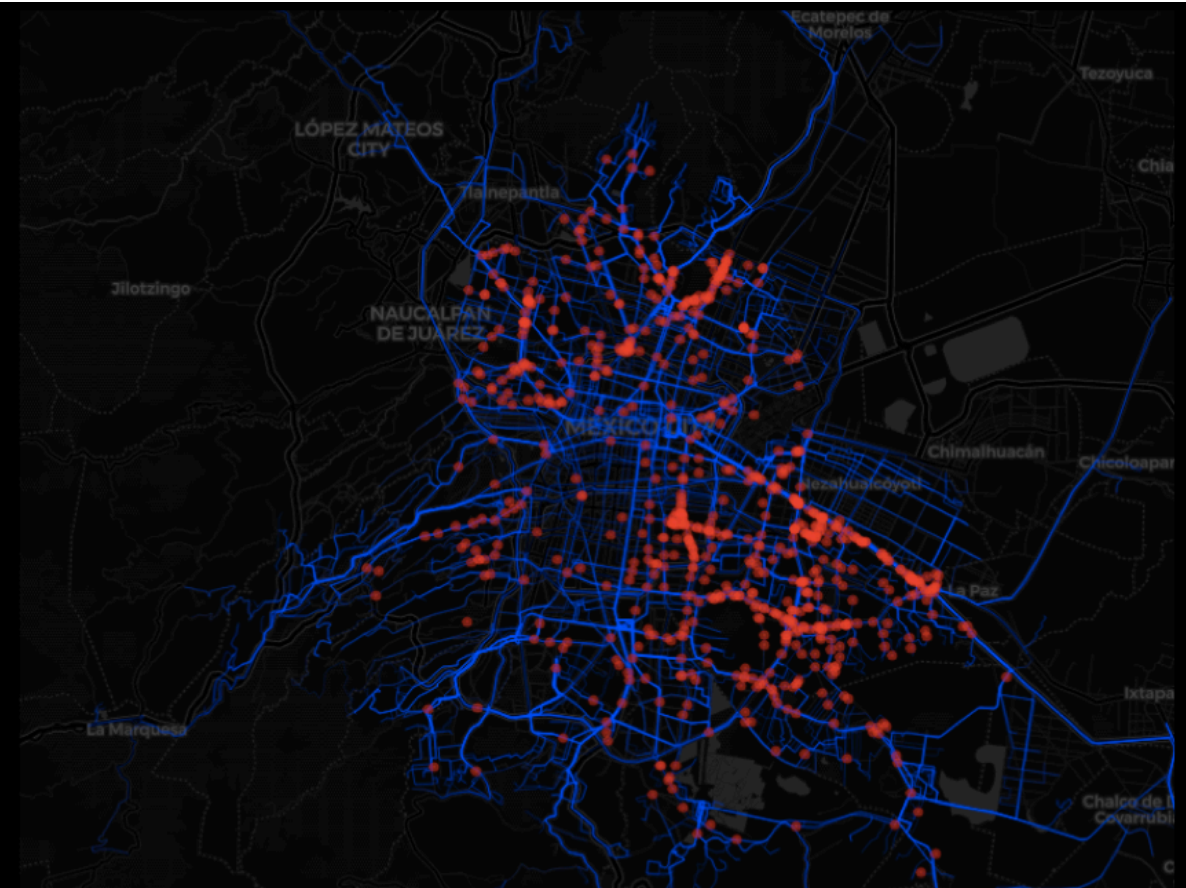
Funciones de *leaflet*



Así como *ggplot* nos permite agregar múltiples geometrías en un solo mapa, igual podemos ir pegando geometrías en un solo mapa. Al igual que *ggplot()*, podemos agregar estas geometrías nuevas especificando la base de geometrías a través del argumento "*data*".

Líneas + puntos =

```
leaflet(datos, options = leafletOptions(zoomControl = FALSE)) %>%
  addProviderTiles("CartoDB.DarkMatter") %>%
  addPolylines(data = rutas,
               weight = 0.3,
               label = rutas$ruta_corre
  ) %>%
  addCircleMarkers(fillColor = "red",
                  color = "red",
                  fillOpacity = 0.3,
                  radius = 0.1
  )
```



Capas de *leaflet*

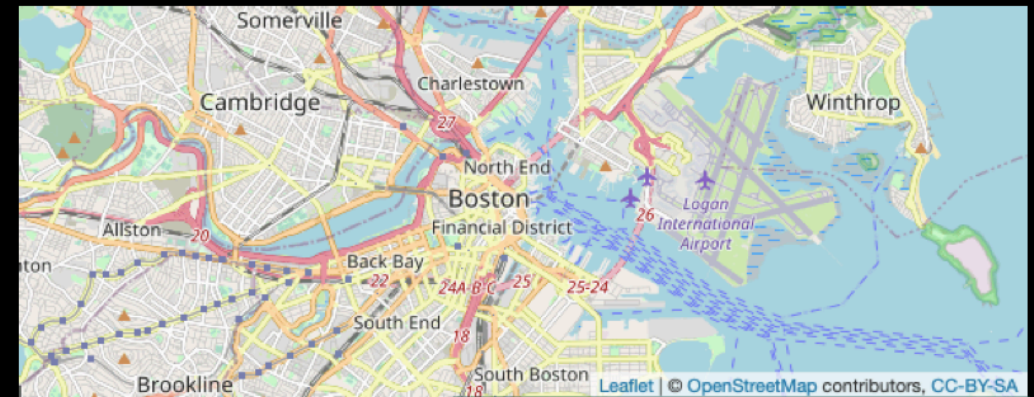
Leaflet maneja distintos tipos de capas:



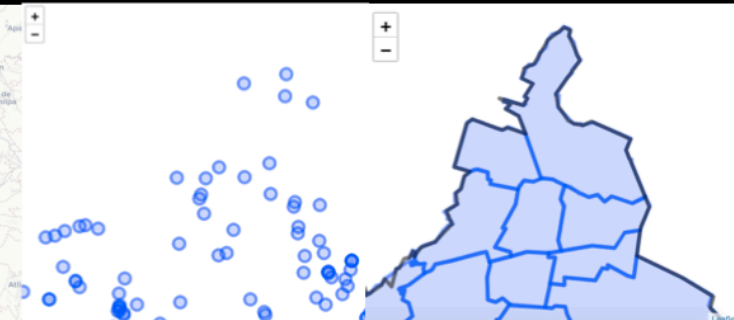
Aportación de los municipios al total de votos obtenidos por Enrique Cárdenas (Coalición PAN-PRD-MC) en la elección Extraordinaria a Gobernador de Puebla (MX) en 2019

Menor al 0.01%
Entre 0.01% y 1%
Entre 1% y 3%
Entre 3% y 5%
45.19 % (PUEBLA)

Leyendas



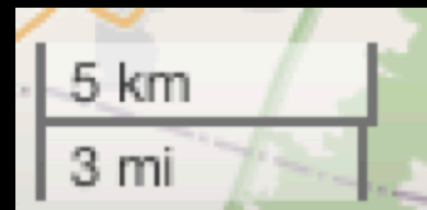
Mapas base



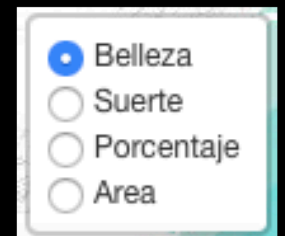
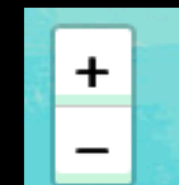
Geometrías



Badges



Información Adicional



Controles

Capas de *leaflet*



Leaflet maneja distintos tipos de capas:

-Mapas base (tiles). Son imágenes georreferenciadas que dan un contexto sobre el lugar donde viven nuestras geometrías.

-Capas de geometrías. Son las geometrías almacenadas en los objetos sí a partir de los cuales vamos a construir nuestros mapas.

-Controles. Los controles son elementos de la interfaz de usuario que controlan la interacción del usuario con la información vertida en el mapa. Entre estos controles destacan los botones de zoom y los botones de control de capas.

-Leyendas. Elementos del mapa que informan acerca de la paleta de colores que estamos usando.

-Información adicional. Leaflet nos permite añadir elementos adicionales a nuestros mapas. Un ejemplo muy usado es la barra de escalas de distancias en el mapa.

-Badges. Leaflet nos permite añadir información adicional personalizada a través de imágenes y texto.

Labels o tooltips



Los tooltips (en {leaflet} también conocidos como **labels**) son elementos de la interfaz de usuario que se despliegan como cajas de texto informativas cuando hacemos *hover* (pasamos el cursor) sobre algún elemento de interés.

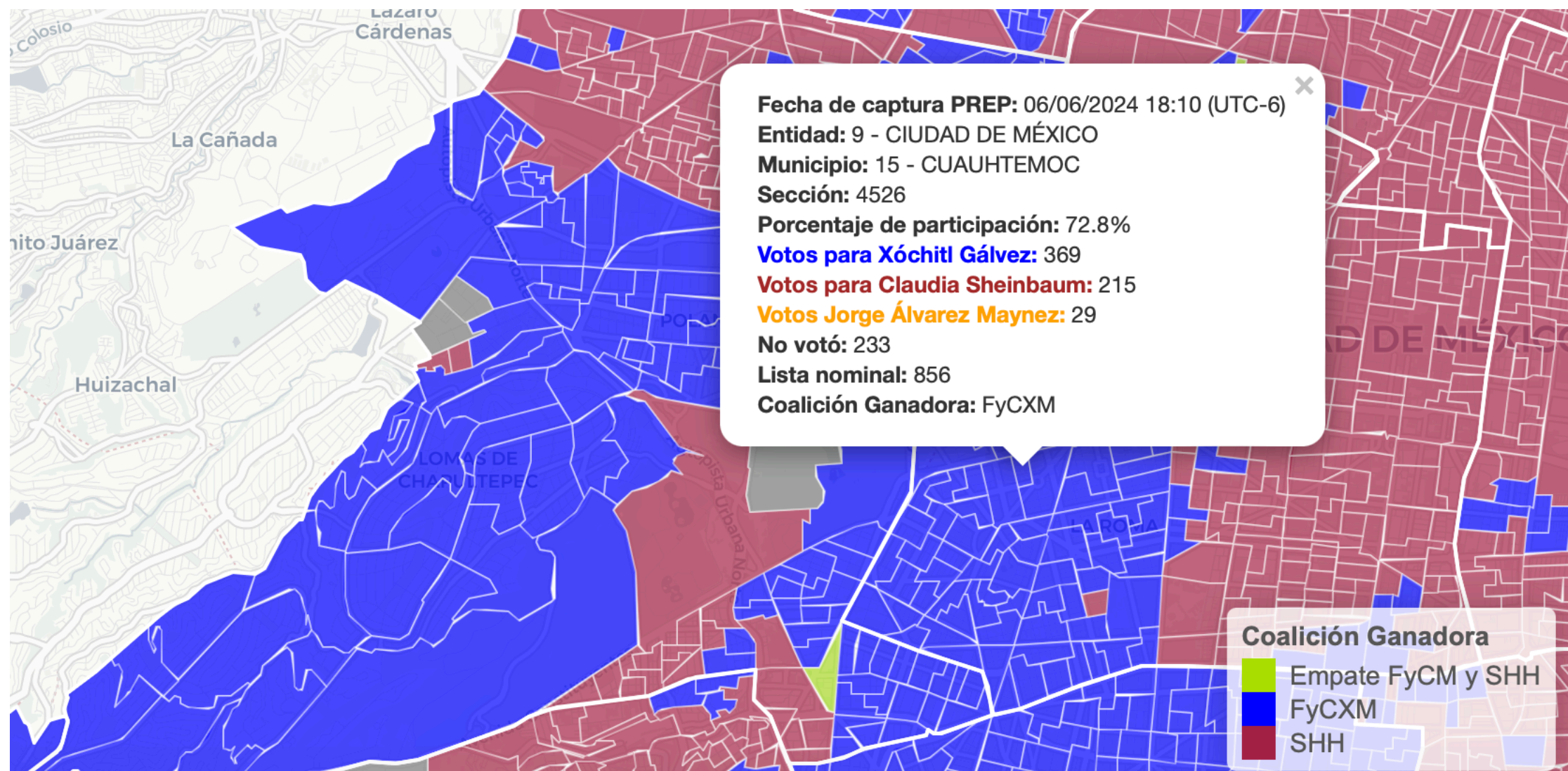


Esto es un
tooltip sobre el
punto **rojo**.

Popups



Los **popups**, en el contexto de leaflet, son elementos de la **interfaz de usuario que despliegan información cuando el usuario lleva a cabo una acción particular** (típicamente un click sobre algún elemento).



Tags de HTML



Algunas etiquetas en **HTML** para dar formato a nuestros textos

1. **<p>** - Párrafo
2. **
** - Salto de línea
3. **** - Negritas
4. **<i>** - Cursiva
5. **<u>** - Subrayado
6. **** - Texto importante (negritas semánticas)
7. **** - Énfasis (cursiva semántica)
8. **<mark>** - Texto resaltado
9. **<small>** - Texto pequeño
10. **** - Texto tachado
11. **<ins>** - Texto insertado (subrayado por defecto)
12. **<sub>** - Subíndice
13. **<sup>** - Superíndice
14. **<blockquote>** - Cita en bloque
15. **<pre>** - Texto preformateado
16. **<code>** - Código de computadora
17. **<hr>** - Línea horizontal
18. **<a>** - Enlace (hipervínculo)
19. **<h1>** a **<h6>** - Encabezados (de mayor a menor jerarquía)

Paletas de colores



Las paletas de colores son conjuntos de colores que dan color a los distintos elementos de un mapa (áreas, líneas, puntos, píxeles). En R, las paletas toman la forma de funciones, las cuales relacionan atributos (datos, valores) con un color correspondiente.

Las paletas varían en función del tipo de datos que queremos plasmar en un mapa.

Tipo de dato	Función generadora de paleta
Numérico continuo, escala continua.	<i>colorNumeric()</i>
Numerico continuo, escala discreta	<i>colorBin() (rangos) o colorQuantile() (cuantiles)</i>
Categorico	<i>colorFactor()</i>

Más info: https://rpubs.com/Juve_Campos/LeafletTutorial



Los argumentos básicos que reciben las funciones de paletas son:

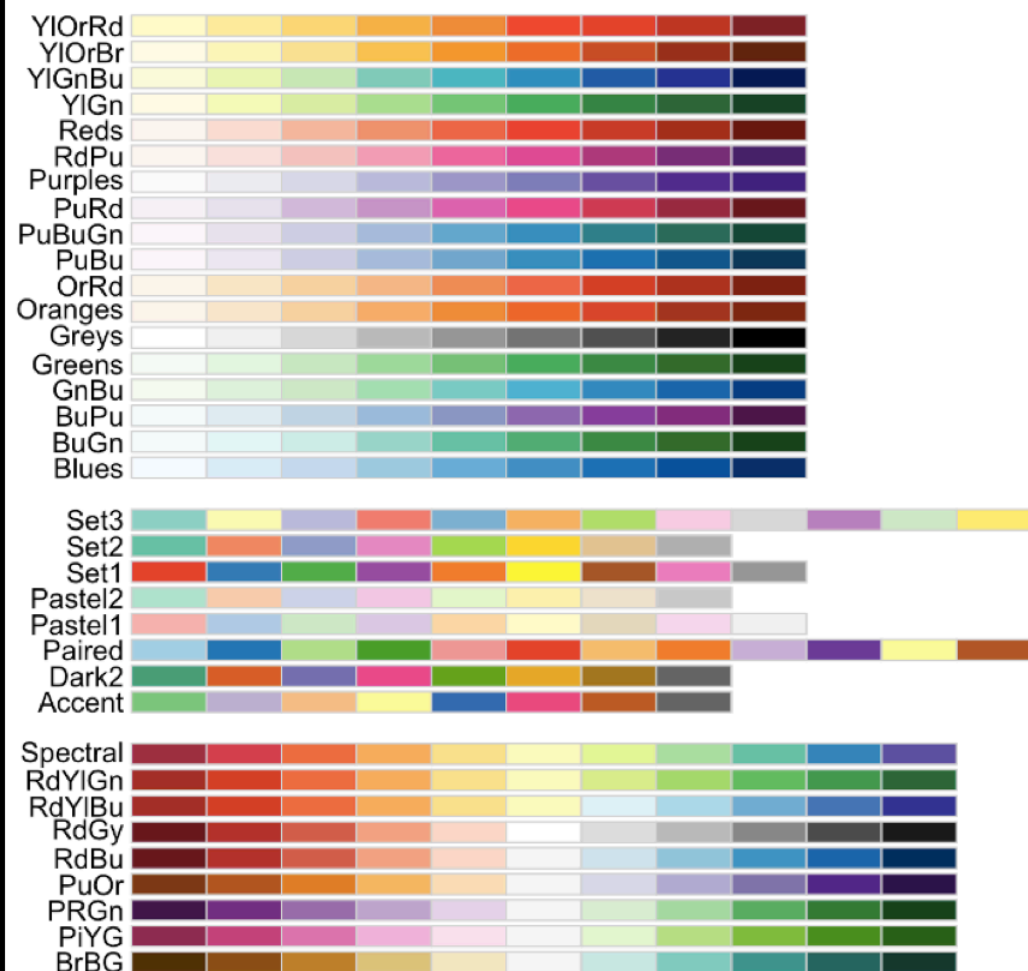
Palette (colores): Este argumento recibe los colores que queremos que conformen a la paleta de colores. Estos pueden ser:

1. Un vector personalizado de colores (en texto o en formato hexadecimal).
2. Un color de la librería “viridis” cargado como cadena de texto: “viridis”, “inferno”, “magma”, “plasma”.
3. Una paleta pre-construida de la librería RColorBrewer, como por ejemplo: “RdYlBu”, “Accent” o “Greens”.
4. Alguna paleta de colores de alguna otra librería (como las famosas paletas de colores de “wespalettes”).

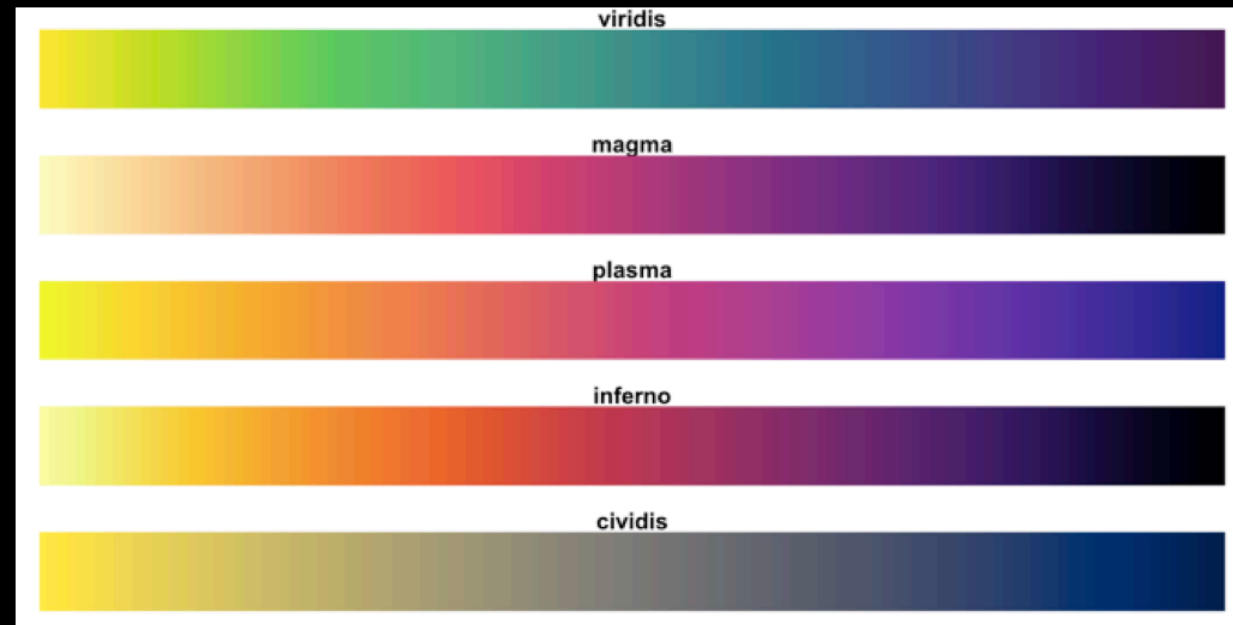
Domain (valores). Este argumento nos solicita el vector de valores a los cuales se les va a asociar un color.

Paletas de colores

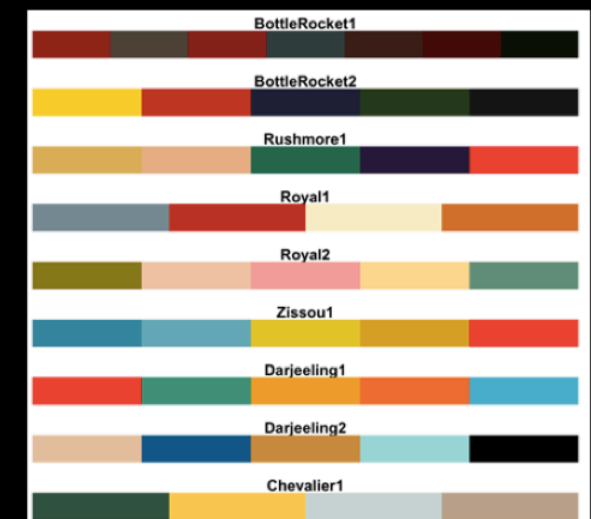
RColorBrewer



Viridis



wespalettes

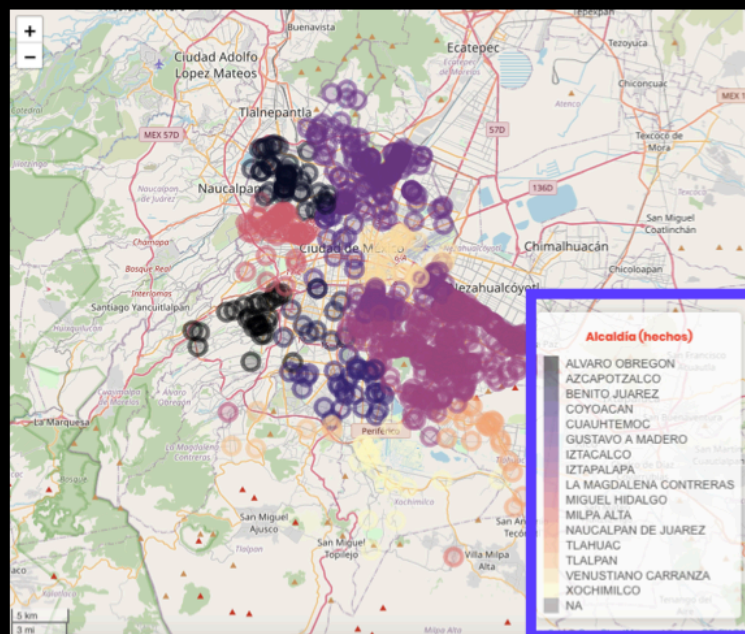


Leyendas

Para añadir leyendas.

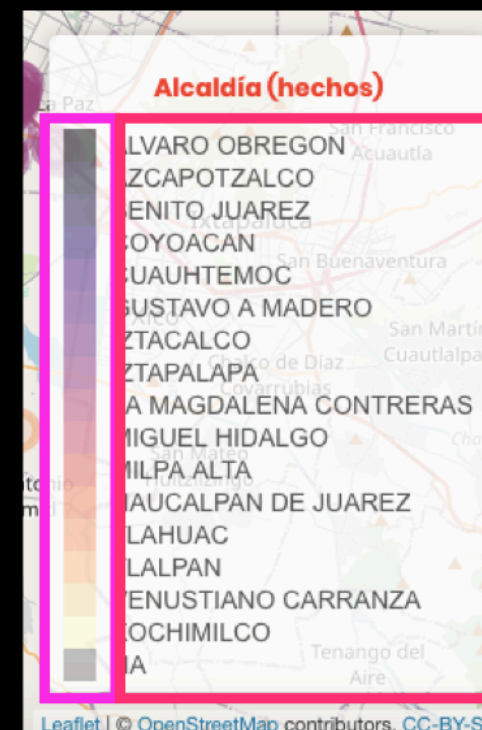
-addLegend(), nos permite construir una leyenda en el mapa. Recibe como argumentos (entre otros):

- la paleta de colores,
- el dominio de los datos,
- la posición de la leyenda dentro del mapa,
- el título de la leyenda (formateable con HTML).



Paleta
(viridis::magma)

Posición
(bottomright)



Título
(formateado
con HTML y CSS)

Dominio
(alcaldías CDMX)

Guardar mapas de leaflet



Para guardar mapas de *leaflet* como HTML y conservar sus capacidades de interactividad, podemos utilizar la función

htmlwidgets::save_widget("nombre_de_archivo.html")

De esta manera obtendremos un documento en HTML que podemos compartir con otros usuarios, o podremos montar en un sitio web.

Una vez que se tiene el HTML, se puede sacar una captura de pantalla de este con la función:

webshot::webshot("map.html", file = "map.png")



Sección práctica

Sección práctica



Elabora los siguientes ejercicios:

1. Elabore una función llamada **nb** (números bonitos :3) que tome como argumentos un numero "x" y un numero "n" y que haga lo siguiente:
 1. Que tome el número x y que lo redondee a "n" decimales.
 2. Que, si al redondear el numero, queda con menos decimales que "n", se rellene con ceros de tal manera que el numero tenga siempre "n" decimales.
 3. Que si es mayor a 1,000, le ponga la coma de separador de miles.

Sección práctica



Elabora los siguientes ejercicios:

2. A partir del código necesario para realizar el mapa de la coalición ganadora en la sesión pasada, elabore una función para generar mapas por entidad.
3. Implemente esa función en un bucle **for** y haga los 32 mapas para cada estado de la república.
4. Elabore nuevamente el mapa de coaliciones ganadoras, pero ahora en la librería **leaflet**. Que la **label** diga la clave del INEGI, el nombre del municipio y el nombre del estado, y que la **popup** diga la coalición ganadora, el total de votos que obtuvo cada candidato presidencial y el porcentaje de participación.
5. Exporte ese mapa en un documento **.html*

Sección práctica



Elabora los siguientes ejercicios:

6. ¿Cómo podría hacer lo que hizo en (3), pero ahora en *leaflet*?
Implemente en un loop **for**

Bases - Sección práctica



Las bases para realizar los ejercicios se encuentran en

- a) el Github oficial de la clase
- b) El drive del Google Classroom